Lekcja: Tablica jednowymiarowa w Pythonie

Lekcja wprowadzająca: zastosowania, deklarowanie, operacje

1. Czym jest tablica jednowymiarowa?

To uporządkowany ciąg elementów, w którym każdy element ma swój numer (indeks). Można ją sobie wyobrazić jako jeden rząd szufladek:

```
indeks:   0     1     2     3     4
        +-----+-----+-----+-----+-----+
        |  5  |  3  |  4  |  2  |  5  |
        +-----+-----+-----+-----+-----+
```

W Pythonie nie ma osobnego typu o nazwie „tablica". Rolę tablicy jednowymiarowej pełnią trzy struktury:

Struktura	Charakter
`list`	podstawowa, najczęściej używana, elastyczna
`array.array`	z biblioteki standardowej, tylko jeden typ liczbowy
`numpy.ndarray`	do szybkich obliczeń na liczbach

Zaczniemy od listy, bo to właśnie jej używa się w 95% przypadków.

2. Po co stosować tablice?

Bez tablicy, żeby zapisać temperatury z tygodnia, trzeba by siedmiu zmiennych:

```
t1 = 18
t2 = 21
t3 = 19
# ...
```

Z tablicą:

```
temperatury = [18, 21, 19, 23, 25, 22, 20]
```

Dzięki temu możesz:

- trzymać wiele wartości pod jedną nazwą,
- przetwarzać je pętlą zamiast powtarzać kod,
- sortować, szukać, sumować, filtrować,
- zmieniać liczbę elementów w trakcie działania programu.

3. Deklarowanie tablicy

Lista

```
pusta = []                            # pusta lista
liczby = [10, 20, 30, 40]             # liczby całkowite
ulamki = [1.5, 2.7, 3.14]             # liczby zmiennoprzecinkowe
imiona = ["Anna", "Piotr", "Kasia"]   # napisy
flagi = [True, False, True]           # wartości logiczne
mieszana = [1, "tekst", 3.14, None]   # różne typy naraz

zera = [0] * 5                        # [0, 0, 0, 0, 0]
zakres = list(range(1, 6))            # [1, 2, 3, 4, 5]
kwadraty = [x * x for x in range(5)] # [0, 1, 4, 9, 16]
```

Tablica array (tylko jeden typ)

```
from array import array

a = array("i", [1, 2, 3, 4])    # "i" = liczby całkowite
b = array("d", [1.5, 2.5])      # "d" = liczby zmiennoprzecinkowe
# a.append("tekst")  -> błąd! Tylko liczby całkowite
```

Zajmuje mniej pamięci niż lista, ale ma mniej możliwości. W praktyce rzadko używana.

Tablica NumPy

```
import numpy as np   # pip install numpy

n = np.array([1, 2, 3, 4])
zera = np.zeros(5)           # [0. 0. 0. 0. 0.]
seria = np.arange(0, 10, 2)  # [0 2 4 6 8]
```

Wszystkie elementy mają ten sam typ, a obliczenia są bardzo szybkie.

4. Indeksowanie

Elementy numerujemy **od zera**:

```
owoce = ["jabłko", "gruszka", "śliwka", "wiśnia", "banan"]

print(owoce[0])     # jabłko   (pierwszy)
print(owoce[2])     # śliwka
print(owoce[-1])    # banan    (ostatni)
print(owoce[-2])    # wiśnia   (przedostatni)
```

Wycinki:

```
print(owoce[1:4])   # ['gruszka', 'śliwka', 'wiśnia']  (od 1 do 3)
print(owoce[:2])    # ['jabłko', 'gruszka']             (pierwsze dwa)
print(owoce[2:])    # ['wiśnia', 'banan']               (ostatnie 2)
print(owoce[::2])   # ['jabłko', 'śliwka', 'banan']     (co drugi)
print(owoce[::-1])  # odwrócona lista
```

Wyjście poza zakres kończy się błędem:

```
print(owoce[10])    # IndexError: list index out of range
```

5. Podstawowe operacje

```
liczby = [4, 7, 1, 9]

# Dodawanie
liczby.append(5)         # [4, 7, 1, 9, 5]   na końcu
liczby.insert(0, 100)    # [100, 4, 7, 1, 9, 5]   na wybranej pozycji
liczby.extend([8, 6])    # dołącza kilka elementów

# Usuwanie
liczby.remove(100)       # usuwa po wartości
ostatni = liczby.pop()   # usuwa i zwraca ostatni
del liczby[0]            # usuwa po indeksie

# Zmiana
liczby[1] = 77

# Informacje
len(liczby)              # liczba elementów
sum(liczby)              # suma
min(liczby)              # najmniejszy
max(liczby)              # największy
liczby.index(77)         # na którym miejscu
liczby.count(1)          # ile razy występuje

# Porządkowanie
liczby.sort()                # sortuje "w miejscu"
liczby.sort(reverse=True)    # malejąco
posortowana = sorted(liczby)  # nowa lista, oryginał bez zmian
liczby.reverse()             # odwraca kolejność
```